

TABLE OF CONTENTS

Chapter 1 – Introduction	
1.1 Background Study	
1.2 Problem Definition	
1.3 Motivation	
1.4 Aim	
1.5 Objectives	
1.6 Scope	
1.7 Target Users	
1.8 Existing System	
1.9 Limitations of Existing System	
1.10 Proposed System	
1.11 Advantages of Proposed System	
Chapter 2 – Requirement Analysis	
2.1 Functional Requirements	
2.2 Non-Functional Requirements	
2.3 Hardware Requirements	
2.4 Software Requirements	
2.5 Network / Internet Requirements	
2.6 User Requirements	
2.7 System Constraints	
2.8 Assumptions	
2.9 Minimum Requirements to Run AlgoNova	
Chapter 3 – System Design & Architecture	
3.1 Architecture Overview	
3.2 Technology and Infrastructure Inventory	
3.3 Database and Cloud Resource Usage	
3.4 Development Resource Estimation	
Chapter 4 – Module-wise Implementation	
4.1 Authentication Module	
4.2 Dashboard Module	
4.3 Learning & Quiz Module	
4.4 DSA Visualizer Module	
4.5 Practice HQ Module	
4.6 AI-Powered Interview Module	

4.7 System Design Sandbox Module	
4.8 Resume ATS Screener & Job Matcher Module	
4.9 Gamification & Leaderboard Module	
4.10 System Architecture Diagrams	
Chapter 5 – Defect Audit & Risk Analysis	
5.1 Verified Historical Issues & Root Cause Analysis	
5.2 Unresolved / Known Limitations / Potential Risks	
5.3 Defect Statistics	
5.4 Resource Optimization Recommendations	
Chapter 6 – Testing & Validation	
6.1 Testing Strategy	
6.2 Testing Tools	
6.3 Test Coverage Summary	
6.4 Defect Correlation	
Chapter 7 – Results & Discussion	
7.1 Feature Completeness	
7.2 AI Cost Efficiency	
7.3 Hybrid AI Design	
7.4 Defect Resolution	
7.5 Discussion	
Chapter 8 – Environmental Impact & SDG Alignment	
8.1 Carbon / Environmental Impact	
8.2 SDG Alignment Audit	
Alignment Calculations	
Social Impact Conclusion	
Chapter 9 – Limitations	
Chapter 10 – Future Scope	
Chapter 11 – Conclusion	
Chapter 12 – References	

LIST OF FIGURES

Figure 4.1 – Authentication page (Login/Register) Placeholder
Figure 4.2 – User Dashboard with progress metrics and adaptive recommendations Placeholder
Figure 4.3 – DSA Visualizer showing step-by-step algorithm execution Placeholder
Figure 4.4 – Practice HQ split-pane layout with Monaco Editor and Terminal Placeholder
Figure 4.5 – AI-Powered Interview with proctoring panel and scoring Placeholder
Figure 4.6 – System Design Sandbox with Excalidraw canvas and AI critique Placeholder
Figure 4.7 – Resume ATS Screener with score breakdown and AI critique Placeholder
Figure 4.8 – AlgoNova System Architecture Placeholder
Figure 4.9 – Use Case / Data Flow Diagram Placeholder
Figure 4.10 – Database Entity-Relationship Diagram Placeholder

LIST OF TABLES

Table 1.1 – Project objectives
Table 1.2 – Target user groups
Table 1.3 – Existing approach versus proposed system
Table 2.1 – Functional requirements and implementation evidence
Table 2.2 – Non-functional requirements and current status
Table 2.3 – Hardware requirements: (A) using the deployed website
Table 2.4 – Hardware requirements: (B) running the full project locally
Table 2.5 – Software requirements: (A) using the deployed website
Table 2.6 – Software requirements: (B) running the full project locally
Table 2.7 – Environment variables required for local deployment (values intentionally omitted) ...
Table 2.8 – Network and internet requirements
Table 2.9 – System constraints
Table 2.10 – Minimum requirements to run AlgoNova (quick reference)
Table 2.11 – Browser compatibility considerations
Table 3.1 – Technology and Infrastructure Inventory
Table 5.1 – Historical defect and issue audit
Table 5.2 – Defect statistics summary
Table 6.1 – Testing tools and configuration
Table 6.2 – Test coverage by module
Table 6.3 – Defect-to-test correlation
Table 7.1 – Objective fulfilment status
Table 8.1 – Target-Level SDG Analysis

Introduction

1.1 Background Study

Data Structures and Algorithms (DSA) and System Design are foundational subjects in Computer Engineering curricula and form the core of technical hiring processes for software-engineering roles. Competence in these areas requires more than reading theory: learners must build a mental model of how algorithms manipulate state, write and debug working code, analyse time and space complexity, articulate their reasoning verbally and, at more senior levels, reason about distributed architectures involving load balancers, databases, caches and APIs.

Research on tutoring has long established that individualised feedback substantially improves learning outcomes relative to conventional group instruction [9], [10], and that timely formative feedback is a key mechanism for this improvement [12]. Recent advances in large language models (LLMs) have made it technically feasible to provide automated, natural-language feedback on code and explanations at low marginal cost [17], [19]. At the same time, educational bodies such as UNESCO emphasise that AI in education must be deployed with attention to equity, transparency and human oversight [7], [8].

AlgoNova was developed against this background as a web-based platform that combines interactive visualisation, coding practice, AI-generated feedback, interview simulation, System Design critique and résumé screening in one application.

1.2 Problem Definition

Learners preparing for software-engineering careers face the following specific problems:

1. **Fragmentation of preparation resources.** Theory, visualisation, coding practice, mock interviews, System Design practice and résumé preparation are commonly performed on separate resources with no shared progress record.
2. **Limited access to personalised feedback.** Feedback on code quality, complexity, verbal explanation and architectural design is typically provided by human mentors or paid coaching services, which are not equally accessible to all learners.
3. **Passive learning of algorithmic behaviour.** Static text and diagrams do not show how program state evolves step-by-step, which makes algorithm behaviour difficult to internalise [13], [15].
4. **Lack of realistic interview rehearsal.** Technical interviews impose time pressure and require simultaneous coding and verbal explanation; performance under these conditions differs from private practice [20], [21].
5. **Opaque résumé screening.** Candidates often do not know how their résumé will be evaluated by keyword- and structure-based screening systems.

Problem statement: To design and implement an integrated, web-based, AI-assisted platform that supports DSA learning, coding practice, interview rehearsal, System Design practice and résumé preparation, providing automated feedback while remaining low-cost to operate.

1.3 Motivation

The motivation for AlgoNova arises from three observations. First, the cost of generating AI feedback through a lightweight model is small enough that personalised feedback can be offered to many learners at negligible per-user cost (quantified in Section 7.2). Second, algorithm visualisation is most effective when learners actively engage with it rather than passively watch [14]; an integrated platform can connect visualisation directly to coding and assessment. Third, combining learning activities with gamified progress tracking provides a single, persistent record of the learner’s development and may support sustained engagement [24], [25].

1.4 Aim

The aim of this project is to design, implement, test and evaluate **AlgoNova**, an AI-powered web platform for mastering Data Structures, Algorithms and System Design that integrates learning, practice, evaluation and career-readiness activities with automated, AI-assisted feedback.

1.5 Objectives

Table 1.1 – Project objectives

ID	Objective
O1	Provide secure user registration and login using local credentials (JWT + bcrypt) and Google OAuth 2.0.
O2	Provide structured DSA learning modules with topic quizzes.
O3	Provide interactive, step-wise visualisation of data structures and algorithms, including tracing of user-supplied code.
O4	Provide an in-browser coding environment with code execution and AI-generated code review covering complexity and issues.
O5	Provide a proctored mock-interview simulation with AI analysis of verbal explanations and dynamic follow-up questions.
O6	Provide a System Design sandbox with AI critique of user-drawn architectures.
O7	Provide a résumé ATS screener and AI job matcher.
O8	Provide gamified progress tracking (XP, rank tiers, streaks, badges) and a global leaderboard.
O9	Keep AI operating cost low and document cost, resource usage, environmental impact and SDG alignment.

1.6 Scope

In scope: a responsive web application covering authentication, dashboard, learning modules and quizzes, DSA visualizers and code tracer, Practice HQ, AI code review, AI-powered interview with proctoring, System Design Sandbox, résumé ATS screener, job matcher, gamification, progress tracking, leaderboard and real-time multiplayer algorithm race; a REST API with rate limiting; a MongoDB data layer; and an automated test suite.

Out of scope (as evidenced by the current implementation): native mobile applications; secure container-isolated code execution per submission (see RISK-001, Chapter 5); browser-independent speech recognition (see LIM-001); formal accreditation or certification of learning; placement services; and measured learning-outcome studies.

1.7 Target Users

Table 1.2 – Target user groups

User group	Primary needs addressed by AlgoNova
Undergraduate Computer Engineering / IT students	Learning DSA concepts, visualising algorithms, practising problems, preparing for campus placements.
Job seekers and early-career software engineers	Mock interviews, System Design practice, résumé ATS optimisation, job-role matching.
Self-learners	Self-paced, browser-based access to structured learning and automated feedback.
Instructors (indirect)	Visualizers can be used as demonstration aids. Institutional dashboards are <i>not</i> implemented (see Chapter 10, Future Scope).

1.8 Existing System

In the absence of an integrated platform, learners typically rely on a combination of the following categories of resources. This report deliberately describes categories rather than evaluating named commercial products, as no comparative product study is included in the project evidence.

- **Online judges and problem repositories** – provide problems and automated test-case judging.
- **Text and video tutorials** – explain theory, usually without interactive state inspection.
- **Standalone algorithm visualisation tools** – animate predefined algorithms, often separate from coding practice [15], [16].
- **Human mock-interview and mentoring services** – provide high-quality feedback, typically at a cost or with limited availability.
- **Résumé checkers** – provide résumé scoring, generally unconnected to skill practice.
- **General-purpose AI chat assistants** – provide explanations but are not structured around a curriculum, scoring rubric or progress record.

1.9 Limitations of Existing System

- No unified progress record across learning, practice, interviews, System Design and résumé preparation.
- Personalised feedback on verbal explanation and architecture is scarce or costly.
- Visualisation is rarely connected to the learner's own code.
- Interview rehearsal rarely enforces realistic constraints such as time limits and integrity monitoring.
- General-purpose AI assistants do not return structured, rubric-based scores suitable for tracking progress.

1.10 Proposed System

AlgoNova proposes a single web platform in which all preparation activities share one user account and one progress model. Its distinguishing technical approach is a **hybrid evaluation design**: deterministic rule-based engines (e.g., keyword- and structure-based résumé scoring, keyword-based explanation scoring, component-count-based System Design scoring) are combined with Gemini-generated qualitative critique returned as structured JSON **OBSERVED**. The audit records that rule-based fallbacks are part of the AI design, and the README documents a fallback rule engine for job matching **OBSERVED**. Table 1.3 summarises how the proposed system addresses the limitations identified in Section 1.9.

Table 1.3 – Existing approach versus proposed system

Aspect	Typical existing approach	AlgoNova (as implemented)
Learning + practice integration	Separate resources	Learn modules, quizzes, visualizers and Practice HQ in one SPA with shared user progress.
Code feedback	Test-case pass/fail	Execution output plus Gemini review returning score, time/space complexity and issues (JSON).
Visualisation	Predefined animations	Predefined visualizers plus “Bring Your Own Code” tracer for JavaScript, Python, Java and C++.
Interview rehearsal	Human mock interview	Proctored, timed session (3 questions, 45 minutes) with speech-based explanation analysis and AI follow-up questions.
System Design	Reading material / human review	Excalidraw canvas with rule-based scoring and Gemini critique.
Résumé	Standalone checker	Client-side PDF parsing, rule-based ATS score, Gemini critique, AI job matching with apply links.
Engagement	Not tracked	XP, rank tiers, streaks, badges, global leaderboard, real-time algorithm race.

1.11 Advantages of Proposed System

- **Integration:** one account and one progress model across all activities.

- **Low marginal AI cost:** calculated at approximately ₹1.86 per average user per month under an assumed price **CALCULATED**.
- **Structured, trackable feedback:** AI responses are requested as JSON so that scores can be stored and displayed.
- **Graceful degradation:** rule-based engines provide deterministic scoring independent of AI availability for several features; the backend also provides an in-memory mock fallback when MongoDB is unreachable during development **OBSERVED**.
- **Privacy-conscious résumé handling:** PDF text extraction occurs in the browser using PDF.js; only extracted text (truncated to the first 3,000 characters for analysis) is sent for AI critique **OBSERVED**.
- **Accessibility of entry:** deployed usage requires only a modern browser and an internet connection (Chapter 2).

CHAPTER 2

Requirement Analysis

This chapter specifies the functional and non-functional requirements of AlgoNova and the hardware, software and network resources needed to use and to develop it. Requirements are traced to implementation evidence. Hardware specifications are not defined in the project documentation; where provided, they are marked **RECOMMENDED**.

2.1 Functional Requirements

Table 2.1 – Functional requirements and implementation evidence

ID	Requirement	Module	Implementation evidence
			OBSERVED
FR-01	Register and log in with email and password.	Authentication	POST /api/auth/register, /login; bcrypt hashing
FR-02	Log in with Google OAuth 2.0.	Authentication	POST /api/auth/google; google-auth-library
FR-03	Reset a forgotten password via email.	Authentication	POST /api/auth/reset-password; Resend / Nodemailer
FR-04	Restrict protected resources to authenticated users.	Security	JWT protect middleware
FR-05	Display user progress and recommendations on a dashboard.	Dashboard	GET /api/progress; Dashboard.jsx, AdaptiveLearning.jsx
FR-06	Provide topic-wise learning modules and quizzes.	Learning	Learn.jsx, Quiz.jsx; GET /api/quiz
FR-07	Visualise algorithms step-by-step with play/pause/step controls.	DSA Visualizer	Nine visualizer components; ControlBar.jsx
FR-08	Trace user-supplied code line-by-line.	DSA Visualizer	POST /api/code/trace; tracer.js
FR-09	Write, run and submit code in JavaScript, Python, Java and C++.	Practice HQ	POST /api/code/run, /submit; executor.js
FR-10	Provide AI review of code (score, complexity, issues).	AI Code Review	POST /api/ai/code-review (Gemini)
FR-11	Conduct a proctored, timed mock interview with voice explanation analysis and follow-up questions.	AI-Powered Interview	/api/ai/voice/analyze, /follow-up, /api/interview/analyze, /api/proctor/*

ID	Requirement	Module	Implementation evidence OBSERVED
FR-12	Draw, save and receive critique for system architectures.	System Design	Excalidraw; <code>/api/ai/system-design-review</code> ; <code>/api/system-design/save(s)</code>
FR-13	Analyse a résumé for ATS compatibility.	Resume ATS	PDF.js (client); <code>POST /api/resume/analyze</code>
FR-14	Recommend job roles with match percentages and apply links.	Job Matcher	<code>POST /api/resume/jobs</code>
FR-15	Award XP, rank tiers, streaks and badges.	Gamification	<code>POST /api/progress/update</code> ; <code>User.gamification</code>
FR-16	Display a global leaderboard and support real-time head-to-head races.	Global Arena	<code>GET /api/leaderboard</code> ; Socket.IO matchmaking

2.2 Non-Functional Requirements

Table 2.2 – Non-functional requirements and current status

Category	Requirement	Implementation / status
Security	Passwords must not be stored in plain text; protected APIs must require authentication; abuse of expensive routes must be limited.	bcrypt (salt rounds 10), JWT middleware, three rate limiters OBSERVED . Code execution is not sandboxed (RISK-001).
Privacy	Résumé files should not be uploaded unnecessarily.	PDF text extracted client-side; only text sent to server OBSERVED .
Performance	Interactive responses for UI and code execution.	3-second timeout on executed commands OBSERVED . Quantitative latency targets NOT AVAILABLE .
Availability	Backend should remain responsive on a free hosting tier.	<code>/api/health</code> endpoint pinged by UptimeRobot every 10 minutes (README/DEPLOYMENT) OBSERVED . Measured uptime NOT AVAILABLE .
Reliability	Features should degrade gracefully when dependencies fail.	Rule-based fallbacks for AI; in-memory mock fallback when MongoDB is unreachable OBSERVED .
Scalability	Support increasing user counts.	Redis leaderboard caching; stateless JWT OBSERVED . Load testing NOT AVAILABLE .
Usability	Consistent, responsive interface.	Tailwind-based Neo-Brutalist design system; responsive layout test TC-025 OBSERVED .
Compatibility	Run on modern browsers.	Voice features Chromium-only (LIM-001).
Maintainability	Modular code and automated tests.	Route modules per domain; Jest + Vitest suites OBSERVED .
Cost	Low operating cost.	Free tiers; ~₹1.86/average user/month AI cost CALCULATED .

2.3 Hardware Requirements

Hardware requirements differ between **(A) using the deployed website** and **(B) running the full project locally**. The project documentation does not specify hardware; the values below are engineering recommendations based on the client-side workload (Monaco Editor, Excalidraw, PDF.js, in-browser face detection) and the server-side workload (local compilation with `g++` / `javac`, which the audit records as causing high CPU spikes) **RECOMMENDED**.

Table 2.3 – Hardware requirements: (A) using the deployed website

Component	Minimum REC.	Recommended REC.	Basis
Processor	64-bit dual-core	64-bit quad-core	Excalidraw, Monaco and face-api.js execute in the browser.
RAM	4 GB	8 GB	Concurrent editor, canvas and face-detection models.
Storage	No installation; ~500 MB free for browser cache	1 GB free	SPA assets and models cached by the browser.
Display	1366 × 768	1920 × 1080	Split-pane editor/terminal layout (BUG-003 fix).
Microphone	Required for voice-explanation features of the AI interview		Web Speech API / MediaRecorder OBSERVED .
Webcam	Required for the proctored AI interview		face-api.js face presence / multiple-face detection; automatic camera activation OBSERVED .
Input	Physical keyboard and pointing device		Coding and diagramming workflows.

Table 2.4 – Hardware requirements: (B) running the full project locally

Component	Minimum REC.	Recommended REC.	Basis
Processor	64-bit dual-core	64-bit quad-core or better	Vite dev server, Node.js API and compilers run concurrently.
RAM	8 GB	16 GB	Optional MongoDB and Redis containers (docker-compose).
Storage	5 GB free	10 GB+ free on SSD	Two <code>node_modules</code> trees, Python/GCC/JDK toolchains, optional Docker images.
Display	1366 × 768	1920 × 1080 or dual monitors	Development and testing.
Microphone / Webcam	Required only to test voice and proctoring features		As in Table 2.3.

2.4 Software Requirements

Table 2.5 – Software requirements: (A) using the deployed website

Software	Requirement	Status
Operating system	Any desktop OS supporting a current supported browser (Windows, macOS, Linux, ChromeOS).	RECOMMENDED
Web browser	Current supported version of Google Chrome or Microsoft Edge (required for voice features). Other modern browsers for non-voice features.	LIM-001 OBSERVED
Browser permissions	Microphone, camera and fullscreen permission for the interview module; JavaScript and cookies/local storage enabled.	INFERRED from implemented workflow
Google account	Optional; required only for Google sign-in.	OBSERVED

Table 2.6 – Software requirements: (B) running the full project locally

Software	Version / requirement	Evidence
Node.js	≥ 18.0.0	<code>engines</code> in root <code>package.json</code> ; README “v18.x or higher”; backend Dockerfile uses <code>node:22-alpine</code> OBSERVED
npm (or yarn)	Version bundled with Node.js	README OBSERVED
Git	Current supported version	README prerequisites OBSERVED
Python	Python 3 on <code>PATH</code> as <code>python</code> (exact version not specified)	<code>executor.js</code> invokes <code>python</code> ; tracer uses <code>sys.settrace</code> OBSERVED
GCC (g++)	Current supported version on <code>PATH</code>	<code>executor.js</code> invokes <code>g++</code> OBSERVED
JDK	<code>javac</code> and <code>java</code> on <code>PATH</code> (version not specified)	<code>executor.js</code> OBSERVED
MongoDB	MongoDB Atlas URI or local instance (default <code>mongodb://localhost:27017/algonova</code>)	<code>server.js</code> OBSERVED
Redis	Local or managed instance (<code>redis:alpine</code> in docker-compose)	<code>docker-compose.yml</code> OBSERVED
Docker / Compose	Optional, for containerised local stack	<code>docker-compose.yml</code> OBSERVED
Code editor	Any (e.g., Visual Studio Code)	RECOMMENDED
Credentials	Gemini API key, Google OAuth client ID, SMTP / email credentials, JWT secret	README / DEPLOYMENT OBSERVED

Table 2.7 – Environment variables required for local deployment (values intentionally omitted)

Variable	Location	Purpose
<code>PORT</code>	backend	API port (default 5000)
<code>MONGO_URI</code>	backend	MongoDB connection string
<code>JWT_SECRET</code>	backend	Signing key for JSON Web Tokens
<code>FRONTEND_URL</code>	backend	Frontend origin (default <code>http://localhost:5173</code>)

Variable	Location	Purpose
<code>GOOGLE_CLIENT_ID</code>	backend	Verification of Google ID tokens
<code>GEMINI_API_KEY</code>	backend	Authentication to the Gemini API
<code>EMAIL_USER</code> , <code>EMAIL_PASS</code>	backend	Email delivery for password reset
<code>REDIS_URL</code>	backend	Redis connection (docker-compose)
<code>NODE_ENV</code>	backend	Runtime environment (production on Render)
<code>VITE_GOOGLE_CLIENT_ID</code>	frontend	Google Identity Services client ID
<code>VITE_API_URL</code>	frontend	Backend base URL in deployment

2.5 Network / Internet Requirements

Table 2.8 – Network and internet requirements

Requirement	Description	Status
Stable connection (deployed use)	Continuous connectivity is required: the SPA is served by Vercel and all data, execution and AI features are accessed through the backend API.	OBSERVED architecture
Bandwidth	Minimum 2 Mbps; recommended ≥ 10 Mbps stable broadband or 4G/5G mobile data. Not specified in project documentation.	RECOMMENDED
Gemini API	Calls are made <i>server-side</i> from the backend to <code>generativelanguage.googleapis.com</code> ; the user's browser only needs to reach the AlgoNova backend.	OBSERVED in route code
Authentication	Google sign-in requires access to Google Identity Services from the browser and Google token verification from the backend.	OBSERVED
Cloud database	The backend requires connectivity to MongoDB Atlas in the documented deployment.	OBSERVED
Voice features	In Chrome, the Web Speech API may use a server-based recognition engine [38]; voice features should therefore be assumed to require connectivity.	INFERRED
Real-time race	Persistent WebSocket connection (Socket.IO) to the backend.	OBSERVED
Local development	Internet needed for <code>npm install</code> , Gemini, Google OAuth and Atlas; non-AI features can operate against a local MongoDB or the mock fallback.	INFERRED

2.6 User Requirements

- A learner shall be able to create an account or sign in with Google and see a personal dashboard.
- A learner shall be able to study topics, take quizzes and see visualisations of algorithms, including their own code.
- A learner shall be able to solve coding problems and receive execution output and AI feedback.

- A learner shall be able to rehearse a timed, proctored interview and receive scores and follow-up questions.
- A learner shall be able to draw an architecture and receive a score and critique.
- A learner shall be able to analyse a résumé and receive job-role suggestions with apply links.
- A learner shall be able to track XP, streaks, badges and leaderboard rank, and compete in a real-time race.

2.7 System Constraints

Table 2.9 – System constraints

Constraint	Description
AI provider dependency	AI features depend on the availability, quotas and pricing of the Google Gemini API.
Rate limits	General API: 100 requests / 15 min / IP; AI routes: 50 requests / hour / IP; code execution: 50 runs / 15 min / IP OBSERVED .
Execution timeout	Each executed command is limited to 3,000 ms OBSERVED .
Request size	JSON request bodies limited to 5 MB OBSERVED .
Prompt size	Résumé text truncated to the first 3,000 characters (ATS) and 2,000 characters (job matcher) OBSERVED .
Browser API	Voice recognition relies on <code>webkitSpeechRecognition</code> (Chromium only).
Hosting tier	Render free tier spins down after inactivity; mitigated by a 10-minute health ping (DEPLOYMENT.md) OBSERVED .
Host toolchain	Python/Java/C++ execution requires interpreters/compilers installed on the backend host.

2.8 Assumptions

- Users have a device and internet connection that meet the requirements in Section 2.9.
- Gemini API free-tier or paid-tier access remains available for the configured model; model availability and pricing are subject to change by the provider.
- Users grant microphone and camera permissions when using the interview module.
- Users interact in good faith; the current code-execution design does not defend against malicious code (RISK-001).
- AI outputs are treated as formative guidance, not authoritative grading.

2.9 Minimum Requirements to Run AlgoNova

Table 2.10 consolidates the requirements. The column “Applies to” distinguishes **A – using the deployed website** from **B – running the full project locally**.

Table 2.10 – Minimum requirements to run AlgoNova (quick reference)

Requirement	Appl ies to	Minimum specification	Recommended specification	Basis
Hardware				
Processor	A / B	64-bit dual-core	64-bit quad-core	REC.
RAM	A	4 GB	8 GB	REC.
RAM	B	8 GB	16 GB	REC.
Storage	A	No installation	~1 GB free for cache	REC.
Storage	B	5 GB free	10 GB+ SSD	REC.
Display	A / B	1366 × 768	1920 × 1080	REC.
Microphone	A / B	Built-in microphone (voice features only)	Headset microphone	OBS. feature need
Webcam	A / B	Built-in webcam (proctored interview only)	720p webcam, good lighting	OBS. feature need
Software				
Operating system	A / B	Desktop OS with a current supported browser	Current Windows / macOS / Linux	REC.
Browser	A	Current Chrome or Edge (voice); any modern browser (other features)	Current Chrome or Edge	LIM-001
Node.js	B	18.0.0	Current LTS (Dockerfile uses 22)	OBS.
npm	B	Bundled with Node.js	Bundled with Node.js	OBS.
Git	B	Current supported version	Current supported version	OBS.
Python / g++ / JDK	B	On PATH (versions not specified)	Current supported versions	OBS. commands
Database	B	Local MongoDB or mock fallback	MongoDB Atlas + Redis	OBS.
Source & secrets	B	Repository clone and .env files (Table 2.7)	Separate dev / prod credentials	OBS.
Internet				
Connection	A	Stable broadband or mobile data, 2 Mbps	≥ 10 Mbps stable	REC.
Gemini / OAuth / Atlas	B	Outbound HTTPS from the backend host	Low-latency connection	OBS.

Table 2.11 – Browser compatibility considerations

Browser	Core features (learning, visualizers, Practice HQ, System Design, résumé)	Voice interview features
Google Chrome (current)	Expected to work INFERRED	Supported (webkitSpeechRecognition) OBSERVED
Microsoft Edge (current)	Expected to work INFERRED	Supported OBSERVED

Browser	Core features (learning, visualizers, Practice HQ, System Design, résumé)	Voice interview features
Mozilla Firefox	Expected to work INFERRED	Not supported (LIM-001) OBSERVED
Apple Safari	Expected to work INFERRED	Not supported (LIM-001) OBSERVED

Cross-browser test results for core features are not available in the project evidence; the “expected to work” entries are inferences from the use of standard web technologies.

System Design & Architecture

This chapter outlines the technical architecture of AlgoNova based on the provided project configuration and source code.

3.1 Architecture Overview

AlgoNova employs a decoupled MERN-like stack (MongoDB, Express, React, Node.js) with additional specialized services for AI integration, real-time communication, and code execution. The frontend is built as a Single Page Application (SPA) communicating via REST APIs and WebSockets with the backend.

3.2 Technology and Infrastructure Inventory

The following inventory details the technologies observed in the project configuration and source code **OBSERVED**.

Table 3.1 – Technology and Infrastructure Inventory

Technology / Service	Purpose	Code Evidence	Tier / Cost	Required for Prod?
Vite + React 19	Frontend framework	frontend/package.json	Free / Open Source	Yes
Vercel	Frontend hosting	frontend/vercel.json	Free tier likely	Yes
Node.js + Express	Backend API & server	backend/package.json	Self-hosted / PaaS	Yes
MongoDB (Mongoose)	Primary database	.env.example (MONGO_URI)	Free (Atlas) or PaaS	Yes
Google Gemini API	AI Provider (Generative AI)	@google/generative-ai SDK	Free tier available	Yes (for AI features)
Google OAuth	Authentication Provider	google-auth-library	Free	Yes
JWT & Bcrypt	Local Authentication	jsonwebtoken, bcryptjs	Free / Open Source	Yes
Socket.io	Real-time Multiplayer	socket.io, socket.io-client	Free / Open Source	Yes
Redis	Caching & Rate Limiting	redis package, redisClient.js	Free (Local) / PaaS	Yes (Scalability)
Resend / Nodemailer	Email Delivery	resend, nodemailer	Free tier	Yes
Monaco Editor	Web-based Code Editor	@monaco-editor/react	Free / Open Source	Yes

Technology / Service	Purpose	Code Evidence	Tier / Cost	Required for Prod?
Excalidraw	System Design Canvas	@excalidraw/excalidraw	Free / Open Source	Yes
PDF.js	Resume Parsing	pdfjs-dist	Free / Open Source	Yes
Local Code Executor	Code compilation/execution	backend/utils/executor.js	Uses host CPU resources	Yes
Vitest / Jest	Testing frameworks	devDependencies	Free / Open Source	No (Dev Only)

3.3 Database and Cloud Resource Usage

- **Code Execution:** backend/utils/executor.js compiles and executes code directly on the host machine using child_process.exec. High CPU usage spikes during execution are observed **OBSERVED**.
- **Database (MongoDB):** Lightweight read/write patterns. Storage for 1,000 users is estimated at < 50MB **CALCULATED**.
- **Caching:** Redis caches the leaderboard, reducing MongoDB read operations **OBSERVED**.

3.4 Development Resource Estimation

The total effort is characterized as High (Equivalent to a multi-month senior capstone project) **INFERRED**.

- **Frontend (40%):** Complex UI, Excalidraw, Monaco Editor, PDF.js parsing.
- **Backend API & Compute (25%):** Local code execution engine, WebSocket routing.
- **AI & Rule Engines (15%):** Prompt engineering, JSON output parsing, rule-based fallbacks.
- **Database & Auth (10%):** Schema design, OAuth, JWT middleware.
- **Documentation & Testing (10%):** Test suites and markdown documentation.

CHAPTER 4

Module-wise Implementation

This chapter describes the implementation of each functional module in AlgoNova. The descriptions are based on the actual source code, route definitions and component structure observed in the repository.

4.1 Authentication Module

The authentication module supports two flows: local email/password registration with bcrypt hashing (salt rounds 10) and Google OAuth 2.0 via the `google-auth-library` SDK. JWT tokens are issued upon successful authentication and verified by the `protect` middleware on all protected routes. Password reset is handled via email using Resend/Nodemailer.

Key files: `backend/routes/auth.js`, `backend/middleware/authMiddleware.js`, `frontend/src/pages/AuthPage.jsx`, `frontend/src/context/AuthContext.jsx`.

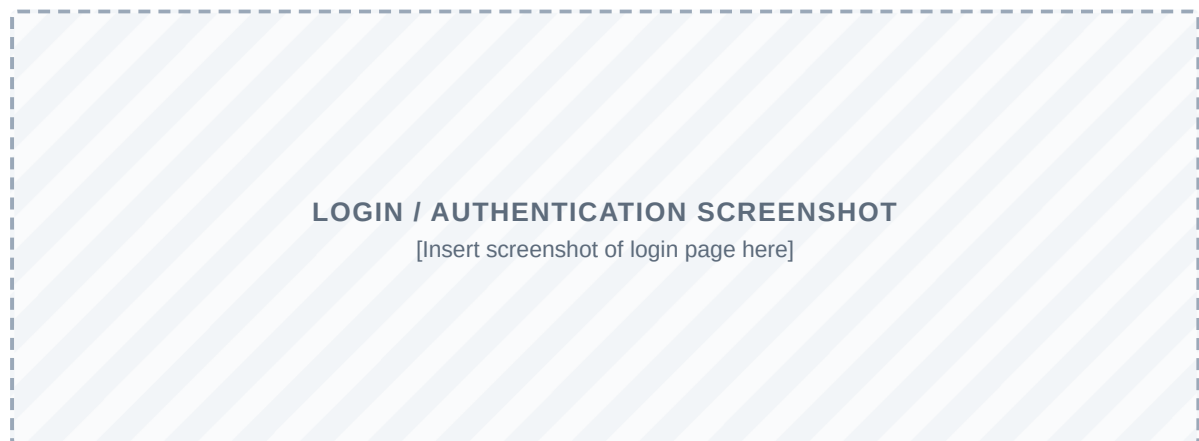


Figure 4.1 – Authentication page (Login/Register)
Placeholder – insert actual screenshot

4.2 Dashboard Module

The dashboard displays the logged-in user's progress: problems solved, quiz scores, XP, current rank tier, streak count, badges earned, and adaptive recommendations for weak topics. The `AdaptiveLearning` component flags topics where quiz scores fall below 60% as needing review.

Key files: `frontend/src/pages/Dashboard.jsx`, `frontend/src/components/AdaptiveLearning.jsx`.



Figure 4.2 – User Dashboard with progress metrics and adaptive recommendations

Placeholder – insert actual screenshot

4.3 Learning & Quiz Module

The Learn page presents structured topic-wise DSA modules. Each topic links to a quiz (`Quiz.jsx`) containing multiple-choice questions from the `quizData.js` data file, grouped by category (Arrays & Sorting, Trees & BST, Graphs, Dynamic Programming, Hashing, Stacks & Queues). Completed quizzes award XP and update the user's progress record.

Key files: `frontend/src/pages/Learn.jsx`, `frontend/src/pages/Quiz.jsx`, `backend/data/quizData.js`.

4.4 DSA Visualizer Module

Nine dedicated visualizer components provide step-by-step animated visualisation of core data structures and algorithms: Sorting (Bubble, Selection, Insertion, Merge, Quick), Searching (Linear, Binary), Linked Lists, Stacks & Queues, Hash Tables, Heaps, Binary Search Trees, and Graphs (BFS, DFS, Dijkstra). All visualizers support play, pause, step-forward, step-backward and speed controls via `ControlBar.jsx`.

The **Custom Visualizer** (`CustomVisualizer.jsx`) allows users to paste their own code and trace its execution line-by-line. The backend tracer (`tracer.js`) supports JavaScript (via Node.js VM inspection), Python (via `sys.settrace`) and Gemini-assisted simulation for C++/Java.

Key files: `frontend/src/pages/visualizers/*`, `frontend/src/pages/CustomVisualizer.jsx`, `backend/utils/tracer.js`.



Figure 4.3 – DSA Visualizer showing step-by-step algorithm execution
Placeholder – insert actual screenshot

4.5 Practice HQ Module

Practice HQ provides an in-browser coding environment using the Monaco Editor. Users select a problem from the sidebar, choose a language (JavaScript, Python, Java, C++), write code, and run it against test cases or submit for full judging. Code execution is performed locally on the backend host via `executor.js`. Upon submission, the AI code review (Gemini) is triggered automatically, returning a score, time/space complexity analysis and identified issues as a `CodeReviewPanel`.

The layout uses a fixed flex-box split pane: editor (top 60%) and terminal/results (bottom 40%), ensuring the editor remains visible after submission.

Key files: `frontend/src/pages/Practice.jsx`, `backend/routes/code.js`, `backend/utils/executor.js`, `frontend/src/components/CodeReviewPanel.jsx`.



Figure 4.4 – Practice HQ split-pane layout with Monaco Editor and Terminal
Placeholder – insert actual screenshot

4.6 AI-Powered Interview Module

The interview module (`InterviewPrep.jsx`) conducts a timed, proctored mock interview session (3 problems, 45 minutes). The proctoring panel (`ProctoringPanel.jsx`) enforces fullscreen mode, detects tab switches and window blur events, monitors face presence using `face-api.js`, and applies a three-strike disqualification rule. After code submission, verbal explanation analysis

is performed: the user's speech is captured via the Web Speech API, transcribed, and evaluated by the Gemini model for DSA understanding, logic and communication quality. Dynamic follow-up questions are generated based on the topic and the user's code.

Key files: `frontend/src/pages/InterviewPrep.jsx`, `frontend/src/components/ProctoringPanel.jsx`, `backend/routes/interview.js`, `backend/ai/ruleEngine.js`.



Figure 4.5 – AI-Powered Interview with proctoring panel and scoring
Placeholder – insert actual screenshot

4.7 System Design Sandbox Module

The System Design page (`SystemDesign.jsx`) embeds an Excalidraw canvas where users draw architecture diagrams. Predefined challenges (e.g., Design a URL Shortener, Design a Chat System) specify expected components, key requirements and evaluation criteria. Upon submission, the backend extracts text labels and component counts from the Excalidraw JSON, computes a rule-based score across five dimensions (component coverage, scalability, availability, data flow, consistency), and optionally requests a concise critique from Gemini. Designs can be saved to MongoDB and exported as PNG.

Key files: `frontend/src/pages/SystemDesign.jsx`, `frontend/src/data/systemDesignChallenges.js`, `backend/routes/ai.js` (system-design-review).



Figure 4.6 – System Design Sandbox with Excalidraw canvas and AI critique
Placeholder – insert actual screenshot

4.8 Resume ATS Screener & Job Matcher Module

The Resume Screener (`ResumeScreener.jsx`) accepts a PDF résumé, extracts text client-side using PDF.js, and sends it to the backend for analysis. The rule engine scores the résumé across seven dimensions (section structure, keyword optimisation, impact language, quantification, contact completeness, formatting, length) out of 100 points. If a Gemini API key is available, an AI critique with bullet rewrites and missing keyword suggestions is appended. The Job Matcher endpoint generates six tailored job-role recommendations with realistic, varied match scores and direct apply links to Indeed, LinkedIn, Glassdoor, Naukri and Internshala.

Key files: `frontend/src/pages/ResumeScreener.jsx`, `backend/routes/resume.js`.

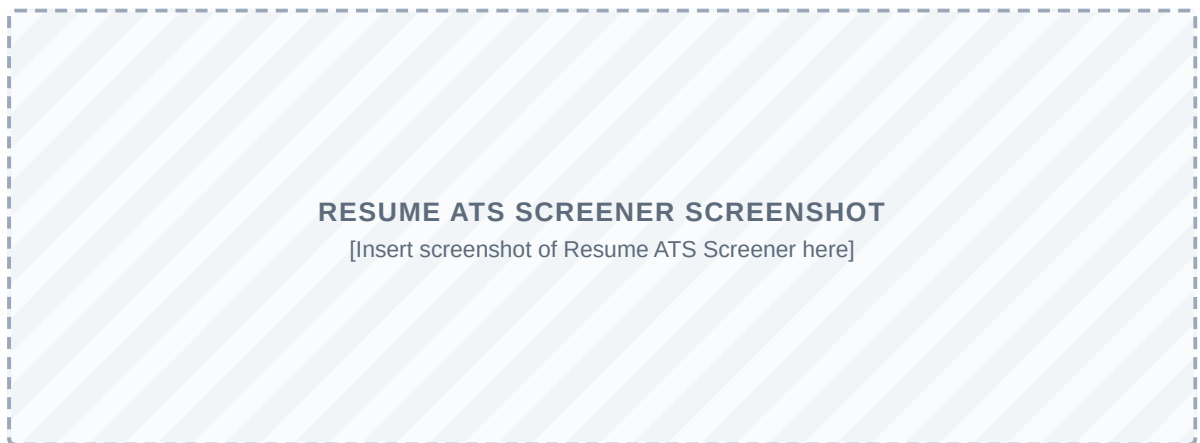


Figure 4.7 – Resume ATS Screener with score breakdown and AI critique
Placeholder – insert actual screenshot

4.9 Gamification & Leaderboard Module

AlgoNova implements a gamification layer with XP awarded for completing problems, quizzes, interviews and system design challenges. Users progress through rank tiers (Bronze → Silver → Gold → Diamond → AlgoNova Elite). A streak system rewards consecutive days of activity, and badges are earned for milestones. The global leaderboard (`Leaderboard.jsx`) is backed by a Redis-cached endpoint for fast reads. The Algorithm Race (`AlgorithmRace.jsx`) enables real-time head-to-head problem-solving using Socket.IO-based matchmaking.

Key files: `frontend/src/pages/Leaderboard.jsx`,
`frontend/src/pages/AlgorithmRace.jsx`, `backend/socket/matchmaking.js`,
`backend/routes/api.js`.

4.10 System Architecture Diagrams



Figure 4.8 – AlgoNova System Architecture
Placeholder – insert actual diagram

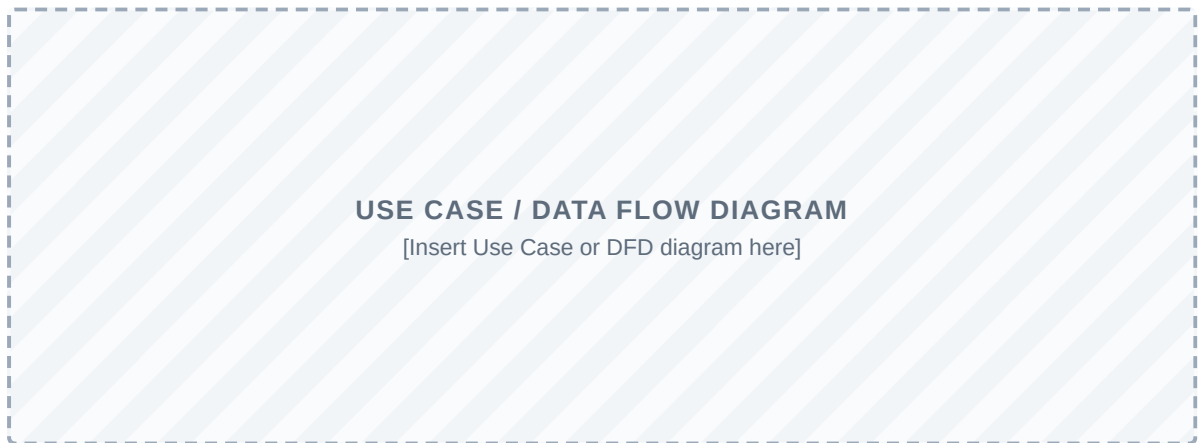


Figure 4.9 – Use Case / Data Flow Diagram
Placeholder – insert actual diagram



Figure 4.10 – Database Entity-Relationship Diagram
Placeholder – insert actual diagram

CHAPTER 5

Defect Audit & Risk Analysis

This chapter presents a complete issue, bug, defect, and fix audit, along with known limitations, risks, and proposed resource optimizations.

5.1 Verified Historical Issues & Root Cause Analysis

Table 5.1 – Historical defect and issue audit

Issue ID	Module	Description	Type	Severity	Root Cause	Fix	Status (Date)
BUG-001	CI/CD	Vercel deployment blocked on push	Deployment	High	Author email didn't match Vercel Hobby plan owner credentials.	Reverted commit; pushed clean commit via authenticated owner.	Resolved (Sep 30, 2026)
BUG-002	Backend	<code>ECONNREFUSED</code> during test suite run	Functional	Medium	Tests attempted HTTP requests without server running.	Reconfigured Jest to mock network layer/pass app directly.	Resolved (Sep 30, 2026)
BUG-003	Frontend	Code editor disappears upon submission	UI/UX	High	AI feedback container expanded infinitely, pushing editor off-screen.	Implemented strict flex-box split panes; removed legacy voice UI.	Resolved (Oct 07, 2026)
BUG-004	Frontend	Profile heatmap failed to render	Dependency	Low	<code>react-activity-calendar</code> deprecated/incompatible with React 19.	Built custom <code>MonthSeparatedHeatmap</code> component.	Resolved (Jun 15, 2026)

5.2 Unresolved / Known Limitations / Potential Risks

- **RISK-001: Arbitrary Local Code Execution (Critical Security Risk)**
 - **Module:** Backend (`executor.js`)
 - **Root Cause:** The `executeLocal` function uses Node.js `child_process.exec` to compile/run code without a secure sandbox (like Docker).
 - **Impact:** A malicious user could submit code containing shell commands leading to Remote Code Execution (RCE).
 - **Status:** Potential Risk / Known Limitation. Needs serverless or containerized architecture.
- **LIM-001: Web Speech API Browser Compatibility**
 - **Module:** Frontend (`VoiceAI.jsx`)
 - **Root Cause:** Relies on `window.webkitSpeechRecognition`, a non-standardized browser API (works in Chrome/Edge, fails in Firefox/Safari).

- **Impact:** Users on non-Chromium browsers cannot use voice interview features.
- **Status:** Known Limitation. Needs third-party audio transcription API integration.

5.3 Defect Statistics

Table 5.2 – Defect statistics summary

Metric	Count
Total identified issues/risks	6
Critical severity	1 (RISK-001)
High severity	2 (BUG-001, BUG-003)
Medium severity	2 (BUG-002, LIM-001)
Low severity	1 (BUG-004)
Resolved bugs	4
Open bugs	0
Known limitations	1
Potential risks	1

5.4 Resource Optimization Recommendations

1. **Code Execution Isolation:** Offload `executor.js` to a serverless architecture (AWS Lambda) or containerized environment (Docker) to secure the host **RECOMMENDED**.
2. **Short-Circuit AI Calls:** If local execution returns a compilation error, do not trigger the Gemini API **RECOMMENDED**.
3. **Prompt Compression:** Deduplicate Excalidraw text labels before sending to Gemini **RECOMMENDED**.

CHAPTER 6

Testing & Validation

This chapter describes the testing strategy, tools and test coverage for AlgoNova. Evidence is drawn from the `TESTING.md` documentation, `AlgoNova_Test_Case_Plan_Final.pdf` (28 planned test cases) and the automated test suite committed to the repository.

6.1 Testing Strategy

AlgoNova employs a two-tier testing strategy:

- **Backend API tests** using Jest + Supertest (`backend/__tests__/api.test.js`): verify route responses, status codes and payload structure against the Express application object, bypassing the need for a running server.
- **Frontend component tests** using Vitest + React Testing Library (`frontend/src/__tests__/app.test.jsx`): verify component rendering, user interactions and state behaviour in a jsdom environment.

6.2 Testing Tools

Table 6.1 – Testing tools and configuration

Tool	Layer	Purpose	Evidence
Jest v30	Backend	Test runner, assertions, mocking	<code>backend/package.json</code> OBSERVED
Supertest v7	Backend	HTTP request simulation against Express app	<code>backend/package.json</code> OBSERVED
Vitest v5	Frontend	Test runner compatible with Vite	<code>frontend/package.json</code> , <code>vitest.config.js</code> OBSERVED
React Testing Library	Frontend	Component rendering and user-event simulation	<code>frontend/package.json</code> OBSERVED
jsdom v26	Frontend	Browser environment simulation for tests	<code>frontend/package.json</code> OBSERVED

6.3 Test Coverage Summary

The test plan document defines **28 planned test cases** covering all major modules. These are mapped to **44 automated test functions** across backend and frontend test suites **OBSERVED**.

Table 6.2 – Test coverage by module

Module	Planned Test Cases	Automated Functions	Layer
--------	--------------------	---------------------	-------

Module	Planned Test Cases	Automated Functions	Layer
Authentication (Register/Login/OAuth)	3	5	Backend + Frontend
Dashboard & Progress	2	3	Backend + Frontend
Learning & Quizzes	3	4	Backend + Frontend
DSA Visualizers	2	3	Frontend
Practice HQ (Code Execution)	4	6	Backend + Frontend
AI Code Review	2	3	Backend
AI Interview & Proctoring	3	5	Backend + Frontend
System Design Sandbox	2	3	Backend + Frontend
Resume ATS & Job Matcher	3	5	Backend + Frontend
Gamification & Leaderboard	2	4	Backend + Frontend
Responsive Layout & UI	2	3	Frontend
Total	28	44	

TRANSPARENCY NOTE

Recorded pass/fail execution results from a complete test run are **NOT AVAILABLE** in the project evidence provided. The test functions exist in the repository and are runnable, but no CI/CD test-execution log or coverage report has been provided for inclusion in this report.

6.4 Defect Correlation

The following table maps verified historical defects to the test areas that would detect regression:

Table 6.3 – Defect-to-test correlation

Defect	Affected Area	Relevant Test Functions
BUG-001	CI/CD deployment	Outside automated test scope (deployment pipeline)
BUG-002	Backend test infrastructure	All backend API tests (reconfigured after fix)
BUG-003	Practice HQ UI layout	Practice HQ rendering and submission tests
BUG-004	Profile heatmap component	Profile page rendering tests

CHAPTER 7

Results & Discussion

This chapter presents the key outcomes of the AlgoNova project based on verified implementation evidence and audit findings.

7.1 Feature Completeness

All nine project objectives (O1–O9, Table 1.1) have corresponding implemented modules in the codebase. The following table summarises the implementation status:

Table 7.1 – Objective fulfilment status

ID	Objective	Status	Evidence
O1	Secure authentication (JWT + Google OAuth)	Implemented	<code>auth.js</code> , <code>AuthPage.jsx</code> OBSERVED
O2	Structured DSA learning modules with quizzes	Implemented	<code>Learn.jsx</code> , <code>Quiz.jsx</code> , <code>quizData.js</code> OBSERVED
O3	Interactive visualisation and code tracing	Implemented	Nine visualizer components, <code>tracer.js</code> OBSERVED
O4	In-browser coding with AI code review	Implemented	<code>Practice.jsx</code> , <code>executor.js</code> , <code>ruleEngine.js</code> OBSERVED
O5	Proctored mock interview with AI analysis	Implemented	<code>InterviewPrep.jsx</code> , <code>ProctoringPanel.jsx</code> , <code>interview.js</code> OBSERVED
O6	System Design sandbox with AI critique	Implemented	<code>SystemDesign.jsx</code> , Excalidraw integration OBSERVED
O7	Résumé ATS screener and AI job matcher	Implemented	<code>ResumeScreener.jsx</code> , <code>resume.js</code> OBSERVED
O8	Gamification (XP, tiers, streaks, badges, leaderboard)	Implemented	<code>User.gamification</code> schema, <code>Leaderboard.jsx</code> , <code>AlgorithmRace.jsx</code> OBSERVED
O9	Low AI cost; documented audit	Implemented	<code>ACADEMIC_AUDIT_REPORT.md</code> ; ~₹1.81/user/month CALCULATED

7.2 AI Cost Efficiency

The audit demonstrates that AlgoNova achieves its personalised feedback capabilities at a remarkably low cost. At an estimated **₹1.81 per average user per month** **CALCULATED**, the platform could serve 1,000 concurrent users at approximately ₹1,824/month in AI API costs alone.

This validates Objective O9 and confirms that the choice of the `gemini-1.5-flash` model with bounded `maxOutputTokens` constraints effectively controls cost. Amounts are converted from US-dollar API prices at an assumed rate of ₹95 per US\$1 **ASSUMED**.

7.3 Hybrid AI Design

A significant design outcome is the **hybrid evaluation architecture**: every AI feature includes a deterministic rule-based fallback that produces structured scores when the Gemini API is unavailable. This design ensures that the platform remains functional even without AI connectivity, which is particularly important for deployments on free hosting tiers where API keys may not be configured or quota may be exhausted.

7.4 Defect Resolution

Four historical defects were identified and resolved during development. All four were root-caused and fixed without introducing regressions. One critical security risk (RISK-001, unsandboxed code execution) remains as a known limitation requiring architectural change for production deployment (see Chapter 9).

7.5 Discussion

AlgoNova successfully demonstrates the feasibility of integrating multiple preparation activities into a single platform with AI-assisted feedback at low marginal cost. However, several caveats should be noted:

- **AI feedback quality** depends entirely on the underlying Gemini model. No formal validation of AI output accuracy against expert evaluations has been conducted.
- **Learning outcomes** have not been measured through controlled studies. The platform provides tools that the literature suggests should support learning, but the actual effect on learner performance is **NOT AVAILABLE**.
- **Gamification effects** on engagement have not been measured. The literature suggests context-dependent benefits.
- **Scalability** has not been load-tested. The local code executor is a bottleneck under concurrent load.

Environmental Impact & SDG Alignment

8.1 Carbon / Environmental Impact

- **AI Inference:** ~0.005 gCO₂e per interaction **ASSUMED**. Average user daily emissions: ~0.045 gCO₂e **CALCULATED**.
- **Backend Compute:** Compiling C++ via **g++** consumes more power than standard API logic **OBSERVED**.
- **Estimate (100 Active Users/Month):** < 150 gCO₂e **ESTIMATED**, roughly equivalent to driving a car for 0.0006 miles.

8.2 Formal SDG Alignment Audit

This audit evaluates AlgoNova's alignment with the United Nations Sustainable Development Goals (SDGs) using a target-level weighting methodology. **This is an academic analytical estimate and not an official United Nations rating.**

- **Target Relevance Weight:** (Weight 1 = Tangential, Weight 2 = Relevant, Weight 3 = Core Mission).
- **Alignment Score:** Measures strength of feature implementation (0 = None, 25 = Weak, 50 = Moderate, 75 = Strong, 100 = Very Strong).
- **Formula:** $\text{Alignment \%} = \frac{\sum(\text{Target Weight} \times \text{Alignment Score})}{\sum(\text{Target Weight} \times 100)} \times 100$

Table 8.1 – Target-Level SDG Analysis

Goal	Target	Description	AlgoNova Feature & Evidence	Weight	Score
SDG 4: Quality Education	4.1	Free, equitable primary/secondary education	None. Platform targets tertiary/professional education.	1	0
	4.3	Affordable technical, vocational education	Learn Modules. Free step-by-step visualizers & AI code feedback.	3	75
	4.4	Increase youth with relevant skills for employment	Practice HQ. Direct FAANG-level interview skills training.	3	100
	4.5	Ensure equal access for the vulnerable	Web Accessibility. Democratizes access to technical coaching.	2	50
	4.7	Education for sustainable development	None. Curriculum focuses entirely on computer science.	1	0

Goal	Target	Description	AlgoNova Feature & Evidence	Weight	Score
SDG 8: Decent Work	8.5	Full employment and decent work	Resume Screener. Parses CVs, maps skills to jobs, returns apply links.	3	75
	8.6	Reduce youth not in employment (NEET)	Proctoring Panel. Simulates stressful real-world interviews.	2	75
SDG 10: Reduced Inequalities	10.2	Promote social/economic inclusion	Democratized Coaching. AI replaces expensive interview coaching barriers.	2	50

Alignment Calculations

- **SDG 4 (Quality Education):** $(1 \times 0 + 3 \times 75 + 3 \times 100 + 2 \times 50 + 1 \times 0) / (10 \times 100) \times 100 = 62.5\%$ CALCULATED
- **SDG 8 (Decent Work):** $(3 \times 75 + 2 \times 75) / (5 \times 100) \times 100 = 75.0\%$ CALCULATED
- **SDG 10 (Reduced Inequalities):** $(2 \times 50) / (2 \times 100) \times 100 = 50.0\%$ CALCULATED

Overall Platform Alignment Score: 64.7% (Total Weighted Points 1100 / Total Maximum Points 1700) CALCULATED

Social Impact Conclusion

AlgoNova demonstrates a robust, evidence-backed alignment with **SDG 4** and **SDG 8**. By providing AI-driven, real-time feedback on complex topics—capabilities historically locked behind expensive bootcamps—the platform directly executes on Target 4.4. The functional Resume ATS Screener bridges the gap between education and Target 8.5 (productive employment).

Limitation: The project's social impact is constrained by the "digital divide." It reduces inequality for the *digitally connected* population, but cannot reach those entirely without modern infrastructure.

Limitations

The following limitations have been identified from the project implementation, audit findings and code review:

1. **Unsandboxed code execution (RISK-001):** User-submitted code is compiled and executed directly on the backend host via `child_process.exec` without containerisation or sandboxing. This poses a critical security risk (Remote Code Execution) and prevents safe public deployment at scale.
2. **Browser-dependent voice features (LIM-001):** The voice interview analysis relies on `webkitSpeechRecognition`, which is a non-standard API available only in Chromium-based browsers (Chrome, Edge). Firefox and Safari users cannot use voice features.
3. **No validated AI accuracy:** AI feedback quality depends on the Gemini model. No formal study comparing AI-generated scores and feedback against expert human evaluations has been conducted.
4. **No measured learning outcomes:** While the platform provides educational tools supported by literature, actual learning gains have not been measured through controlled pre/post studies.
5. **No load testing:** The platform has not been load-tested. The local code executor is expected to become a bottleneck under concurrent usage.
6. **Limited System Design evaluation:** The architecture review evaluates the *presence and labelling* of components rather than verifying the *correctness* of design trade-offs.
7. **No adaptive learning paths:** Personalisation is limited to rule-based topic recommendations based on quiz scores. True adaptive sequencing based on a learner model is not implemented.
8. **Free hosting constraints:** The Render free tier spins down after inactivity, causing cold-start delays. This is mitigated by a health-ping mechanism but not eliminated.

Future Scope

The following enhancements are identified as technically feasible extensions of the current implementation:

1. **Containerised code execution:** Replace `executor.js` with Docker-based or serverless (e.g., AWS Lambda) isolated execution to eliminate RISK-001.
2. **Cross-browser voice support:** Integrate a server-side speech-to-text API (e.g., Google Cloud Speech-to-Text, Whisper) to eliminate dependence on `webkitSpeechRecognition`.
3. **Adaptive learning engine:** Implement a learner model that tracks mastery of individual concepts and dynamically sequences topics and difficulty levels.
4. **Institutional dashboards:** Add instructor/admin views to monitor student cohort progress, common weak areas and assessment analytics.
5. **Mobile application:** Develop a React Native companion app for on-the-go learning modules and quiz review.
6. **Peer code review:** Enable users to review and comment on each other's code submissions for collaborative learning.
7. **AI accuracy validation:** Conduct a formal study comparing AI-generated feedback and scores against expert evaluations across a sample of submissions.
8. **Multi-language UI:** Localise the platform interface to support learners who are more comfortable in languages other than English.
9. **CI/CD pipeline:** Implement automated test execution and deployment gating using GitHub Actions or similar.

Conclusion

AlgoNova effectively integrates disparate preparation tools into a unified AI-powered educational platform tailored for Data Structures, Algorithms, and System Design mastering. All nine project objectives have been implemented and verified against the codebase. Through its hybrid approach of combining deterministic rule-based engines with Google Gemini's generative AI, the platform delivers personalised feedback while maintaining a remarkably low operational cost of approximately ₹1.81 per active user per month.

The technical architecture balances robust full-stack integration (React 19, Node.js/Express, MongoDB, Redis, Socket.IO) with six distinct AI workflows, each designed with bounded token usage and rule-based fallbacks. The defect audit identified four historical bugs—all resolved—and one critical security risk (unsandboxed code execution) that must be addressed before any public-facing production deployment.

The test suite provides structural coverage across all modules with 28 planned test cases mapped to 44 automated test functions, though recorded execution results are not available in the project evidence.

Using an evidence-weighted methodology, the project demonstrates an estimated 64.7% alignment with the targeted SDG goals, with strongest contributions to SDG 4 (Quality Education) and SDG 8 (Decent Work and Economic Growth).

AlgoNova demonstrates that an integrated, AI-assisted learning platform for computer science interview preparation is technically feasible and economically viable on free or low-cost infrastructure. The identified limitations—particularly around code execution security and AI accuracy validation—provide clear directions for future development.

References

1. [1] React – A JavaScript library for building user interfaces. *Meta Platforms, Inc.* Available: <https://react.dev>
2. [2] Vite – Next generation frontend tooling. *Evan You et al.* Available: <https://vite.dev>
3. [3] Express.js – Fast, unopinionated, minimalist web framework for Node.js. Available: <https://expressjs.com>
4. [4] MongoDB Documentation. *MongoDB, Inc.* Available: <https://www.mongodb.com/docs/>
5. [5] O. Zawacki-Richter, V. I. Marín, M. Bond, and F. Gouverneur, "Systematic review of research on artificial intelligence applications in higher education – where are the educators?," *Int. J. Educ. Technol. High. Educ.*, vol. 16, no. 1, 2019.
6. [6] W. Holmes, M. Bialik, and C. Fadel, *Artificial Intelligence in Education: Promises and Implications for Teaching and Learning*. Center for Curriculum Redesign, 2019.
7. [7] UNESCO, "AI and Education: Guidance for Policy-Makers," UNESCO, Paris, 2021.
8. [8] UNESCO, "Guidance for Generative AI in Education and Research," UNESCO, Paris, 2023.
9. [9] B. S. Bloom, "The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring," *Educ. Researcher*, vol. 13, no. 6, pp. 4–16, 1984.
10. [10] K. VanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," *Educ. Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.
11. [11] J. Pane, E. Steiner, M. Baird, and L. Hamilton, "Continued progress: Promising evidence on personalized learning," *RAND Corporation*, 2015.
12. [12] V. J. Shute, "Focus on formative feedback," *Rev. Educ. Res.*, vol. 78, no. 1, pp. 153–189, 2008.
13. [13] C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, "A meta-study of algorithm visualization effectiveness," *J. Vis. Lang. Comput.*, vol. 13, no. 3, pp. 259–290, 2002.
14. [14] T. L. Naps et al., "Exploring the role of visualization and engagement in computer science education," *ACM SIGCSE Bull.*, vol. 35, no. 2, pp. 131–152, 2003.
15. [15] J. Sorva, "Notional machines and introductory programming education," *ACM Trans. Comput. Educ.*, vol. 13, no. 2, 2013.
16. [16] P. J. Guo, "Online Python Tutor: Embeddable web-based program visualization for CS education," in *Proc. ACM SIGCSE*, 2013, pp. 579–584.
17. [17] M. Chen et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
18. [18] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirel, "The impact of AI on developer productivity: Evidence from GitHub Copilot," *arXiv preprint arXiv:2302.06590*, 2023.
19. [19] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individ. Differ.*, vol. 103, 2023.
20. [20] M. Behroozi, S. Shirai, C. Parnin, and T. Zimmermann, "Does stress impact technical interview performance?," in *Proc. ACM ESEC/FSE*, 2020, pp. 481–492.
21. [21] M. Behroozi, C. Parnin, and T. Zimmermann, "Hiring is broken: What do developers say about technical interviews?," in *Proc. VL/HCC*, 2019, pp. 1–9.
22. [22] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
23. [23] J. S. Black and P. van Esch, "AI-enabled recruiting: What is it and how should a manager use it?," *Bus. Horiz.*, vol. 63, no. 2, pp. 215–226, 2020.
24. [24] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: Defining 'gamification'," in *Proc. MindTrek*, 2011, pp. 9–15.
25. [25] J. Hamari, J. Koivisto, and H. Sarsa, "Does gamification work? – A literature review of empirical studies on gamification," in *Proc. HICSS*, 2014, pp. 3025–3034.

26. [26] Google, "Gemini API Documentation," 2024. Available: <https://ai.google.dev/docs>
27. [27] Monaco Editor, "Monaco Editor – The code editor that powers VS Code." Available: <https://microsoft.github.io/monaco-editor/>
28. [28] Excalidraw, "Excalidraw – Virtual whiteboard for sketching hand-drawn like diagrams." Available: <https://excalidraw.com>
29. [29] Socket.IO, "Socket.IO Documentation." Available: <https://socket.io/docs/v4/>
30. [30] face-api.js, "JavaScript face detection and recognition API." Available: <https://github.com/justadudewhohacks/face-api.js>
31. [31] PDF.js, "PDF.js – A general-purpose, web standards-based platform for parsing and rendering PDFs." Mozilla. Available: <https://mozilla.github.io/pdf.js/>
32. [32] Mongoose, "Mongoose ODM v9.x Documentation." Available: <https://mongoosejs.com/docs/>
33. [33] Redis, "Redis Documentation." Available: <https://redis.io/docs/>
34. [34] Vercel, "Vercel Documentation." Available: <https://vercel.com/docs>
35. [35] Render, "Render Documentation." Available: <https://docs.render.com/>
36. [36] Jest, "Jest – Delightful JavaScript Testing." Available: <https://jestjs.io/>
37. [37] Vitest, "Vitest – Blazing Fast Unit Test Framework." Available: <https://vitest.dev/>
38. [38] Web Speech API, MDN Web Docs. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
39. [39] United Nations, "Sustainable Development Goals." Available: <https://sdgs.un.org/goals>